REINFORCEMENT LEARNING FOR CHIP PLACEMENT PROBLEM UNDERSTANDING AND PLACEMENT REFINEMENT

MID-SEMESTER PROJECT REPORT

Aditya Birari
2023A3PS0700P
EEE Department

Pratyush Mohata
2023A3PS0690P
EEE Department

ABSTRACT

Macro placement determines the physical locations of large circuit blocks and influences the quality of subsequent standard-cell placement and routing. This project studies reinforcement learning (RL) as a means of refining an existing layout. The mid-semester objective is to establish the problem formulation, understand relevant literature, identify a defensible research question, and implement an initial experimental scaffold. We review constructive RL, placement refinement, offline transfer, and diffusion-based placement. Since existing refinement research already evaluates unseen circuits, we propose a narrower study: whether a frozen refinement policy remains useful when both the circuit and its starting-layout distribution change. The planned contribution is a controlled robustness extension evaluated against strong search baselines under explicit compute budgets. As initial implementation, we develop a synthetic placement environment, weighted wirelength evaluator, legality masks, and a trainable linear REINFORCE policy. Measurements on four held-out synthetic netlists demonstrate the operation of the scaffold and the strength of a greedy baseline. These measurements do not establish real-circuit generalization, reproduction of the reference method, or power–performance–area improvement.

1 Problem understanding and motivation

1.1 Placement within the circuit-design flow

Digital design begins with a functional description and a netlist specifying components and their connections. Floorplanning establishes the core outline, large blocks, and physical constraints. Placement assigns physical coordinates to instances; subsequent clock-tree synthesis and routing connect them. A *macro* is a relatively large block, such as a memory instance, whereas standard cells implement smaller logic functions. Our scope is the physical placement of existing blocks. Their logical functionality, internal circuitry, and dimensions remain fixed.

Macro locations influence the space available for standard cells and interconnect. Shorter connections can be desirable, but concentrating blocks in one region may obstruct routing. A useful placement must satisfy geometry and fixed-object constraints while balancing several imperfectly aligned objectives. Analytical placement tools provide strong starting solutions and will serve as the backbone of the eventual real-circuit study (DREAMPlace contributors, 2026).

1.2 Mathematical representation and constraints

Represent the circuit as a hypergraph $G = (V, E)$: vertices correspond to components and a hyperedge represents a net connecting two or more pins. Let macro i have lower-left coordinate (x_i, y_i), fixed dimensions (w_i, h_i), and pin offsets $(\delta_{ip}^x, \delta_{ip}^y)$. The physical pin location is $(x_i + \delta_{ip}^x, y_i + \delta_{ip}^y)$. A placement P assigns coordinates to all movable macros.

For a die of width W and height H_d, boundary constraints include

$$0 \le x_i \le W - w_i, \qquad 0 \le y_i \le H_d - h_i. \tag{1}$$

For each pair of rectangles i, j, at least one separation condition must hold:

$$x_i + w_i \le x_j \ \vee \ x_j + w_j \le x_i \ \vee \ y_i + h_i \le y_j \ \vee \ y_j + h_j \le y_i. \tag{2}$$

Real designs additionally impose blockages, fixed instances, legal orientations, halos, site rules, and technology-specific restrictions. Our initial implementation supports a deliberately smaller set: integer-grid rectangles, fixed terminals, and one fixed blockage.

If N macros each have K candidate locations, a crude unconstrained assignment count is K^N. Constraints reduce the admissible set but introduce coupled decisions. The feasible placement set is discrete or nonconvex; a move improving one net can worsen several others.

1.3 Wirelength, congestion, regularity, and PPA

Weighted half-perimeter wirelength is

$$H(P) = \sum_{e \in E} \omega_e \left[\max_{p \in e} p_x - \min_{p \in e} p_x + \max_{p \in e} p_y - \min_{p \in e} p_y \right], \quad \omega_e \ge 0. \tag{3}$$

For pins $(1,1), (4,2), (3,5)$ on one unweighted net, HPWL is $3 + 4 = 7$. Moving the second pin to $(3,2)$ reduces it to $2 + 4 = 6$. This does not describe the effect on other incident nets or actual routed paths. With affine pin translations, HPWL is convex but nonsmooth in unrestricted coordinates; the non-overlap constraints make the overall placement problem nonconvex.

Congestion compares routing demand with available routing resources. A fast geometric estimator, global-route overflow, and detailed-route violations are distinct measurements. *Regularity* expresses a preference for organized macro geometry, often proximity to boundaries; it is a proxy rather than a routing guarantee. *PPA* denotes power, performance, and area. Timing metrics such as worst negative slack and total negative slack require a specified physical flow and constraints. Power requires libraries and activity assumptions. Repositioning fixed-size blocks on a fixed die does not itself demonstrate area reduction (The OpenROAD Project, 2026).

1.4 Why refine an existing placement?

A constructive agent places blocks before all future positions are known. A refinement agent starts from a complete arrangement and modifies it. It can therefore evaluate a move against more complete geometric context. This motivates denser and more informative feedback, but dense rewards also exist in constructive methods. The distinction is the available placement information, not a universal division between sparse and dense rewards.

Refinement can still fail: the starting arrangement may lie in a poor optimization basin, the proxy objective may disagree with routing, or useful changes may require coordination across multiple macros. A strong initial layout can also leave little improvement headroom. These observations motivate measuring when refinement is useful, rather than assuming that every intervention helps.

2 Literature review and research gap

2.1 Constructive placement and transferable policies

Lai et al. (2022) introduce pixel-based placement representations with action masks and dense wirelength feedback. This establishes that masking and frequent rewards predate the refinement formulation. Lai et al. (2023) use an offline decision transformer to transfer knowledge from placement trajectories and support adaptation to unseen circuits. These methods motivate transferable representations while requiring us to distinguish frozen-weight evaluation from target-circuit fine-tuning.

Classical refinement is also relevant. Pu et al. (2024) combine incremental macro adjustment, peripheral guidance, and legalization while preserving useful placement relationships. Consequently, improvement over an initial layout does not by itself show that learning is responsible; non-RL refinement must be a serious control.

2.2 MaskRegulate: the principal reference method

Xue et al. (2024) formulate RL as a regulator of an existing placement. A selected macro is reassigned to a grid location using placement, wirelength, and regularity masks. The method uses PPO and evaluates global HPWL

after a standard-cell placement stage. Its generalization experiment trains on `superblue1/3/4/5` and evaluates `superblue7/10/16/18`; therefore, unseen-circuit testing is already part of the literature.

An important detail is the OLD/NEW occupancy representation. Its mask excludes macros already adjusted during the current pass, while unadjusted old locations can remain available. Intermediate overlap is therefore possible. The final proposal must preserve this behavior when reproducing the reference and independently validate complete-pass outputs. The baseline already normalizes quantities; a new scaling convention must be compared with that native convention, not with an invented unnormalized baseline.

Our source inspection also motivates validation-only checkpoint selection and explicit seeding before model construction. The released training routine can use evaluation scores to save best checkpoints. This is a code-path risk to avoid in our harness, not a claim about how the published experiments were conducted (LAMDA-BBO, 2026).

2.3 ChipDiffusion and newer placement approaches

Lee et al. (2025) learn a graph-conditioned diffusion model from synthetic circuit–layout pairs. The model denoises coordinates and uses guidance and legalization to improve placement. Frozen-weight transfer can therefore still involve substantial optimization at inference. A full placement generator and an existing-layout refiner have different input information; their published tables cannot be ranked as a controlled head-to-head comparison.

Recent work further narrows the novelty boundary. EXPlace incorporates expert guidance and timing feedback (Gao et al., 2026), while FlowPlace uses flow matching and hard-constraint sampling (Xie et al., 2026). We do not claim that expert priors, synthetic-data training, boundary preference, or constraint handling are new. These papers provide current context; the core implementation target remains MaskRegulate, with pretrained diffusion comparison conditional on interface compatibility.

Table 1: Literature synthesis and implications for the proposed study. These are method distinctions, not a numerical ranking across incompatible experiments.

Work	Relevant idea	Implication for this project
MaskPlace (2022)	Constructive RL with dense masks	Dense rewards and masking are established.
ChiPFormer (2023)	Offline sequence-model transfer	Separate frozen transfer from adaptation.
IncreMacro (2024)	Non-RL placement refinement	Compare against strong classical refinement.
MaskRegulate (2024)	PPO adjustment of an existing layout	Cross-circuit refinement is already evaluated.
ChipDiffusion (2025)	Synthetic-trained coordinate generation	Match whole pipelines and inference costs.
EXPlace / FlowPlace (2026)	Expert guidance / constrained generation	Scope novelty beyond these existing ingredients.

2.4 The specific gap we will investigate

The proposed question is whether a frozen refiner transfers *useful improvement* when both the netlist and the distribution of starting placements change. A policy might learn general geometric improvement rules, or it might learn to correct errors characteristic of its training initializer. Testing only familiar initializer conditions cannot distinguish these explanations. Starting-layout quality is a separate confound: a large gain from a weak start is easier to obtain than improvement of a well-optimized placement.

Our intended contribution is a controlled study of these effects and a modest robustness extension. We do not assert a first-ever generalization result or an entirely new placement algorithm. The experimental contribution depends on clean splits, strong controls, ablations, and an explanation of failures as well as successes.

3 Problem formulation and proposed research direction

3.1 Input, output, and transfer setting

The input is a circuit hypergraph, macro dimensions and pins, fixed geometry/constraints, and a feasible initial placement P_0. The output is a checked feasible placement P^* within a declared deployment budget. Policy weights are frozen on final test circuits. Evaluating candidate costs on a test netlist is part of deployment; updating policy weights on that netlist is a separate adaptation experiment.

For the real-circuit experiment, all variants of one parent design remain in the same split. Training designs fit weights; validation designs select checkpoints and hyperparameters; final test designs are used only for the reported evaluation. New seeds or perturbed layouts of one netlist are not independent circuit families.

3.2 Refinement as an MDP

The full environment state contains the graph, geometry, placement, selected macro, pass/order phase, and horizon context. A neural policy may receive masks or compact features derived from that state; we do not assume those observations are provably Markov-sufficient. Table 2 specifies the intended formulation.

Table 2: Refinement MDP and its relation to the initial prototype.

Element	Intended reference study	Initial synthetic implementation
State/observation	OLD/NEW maps, current macro, geometry and cost masks	Full rectangle environment; five candidate features for the policy
Action	Mask-permitted grid destination	Feasible integer-grid destination, including current location
Transition	Update a macro and passwise masks	Update one macro while preserving rectangle legality
Reward	Declared wirelength/regularity terms	Normalized decrease in weighted center-pin HPWL
Endpoint	Independently checked completed pass	Every step is feasible; two passes per episode
Learning	Native PPO reference and controlled extensions	Linear softmax policy trained with REINFORCE

A proposed scalar cost uses fixed per-episode reference scales:

$$J(P; P_0) = \alpha \frac{H(P)}{s_H(P_0)} + \beta \frac{R(P)}{s_R(P_0)}, \qquad s_H, s_R > 0. \tag{4}$$

Here R is a specified regularity penalty. Zero or near-zero references require fixed, declared fallback scales. We first preserve the native objective; adding a congestion term would be an additional experiment. The initial prototype uses only H.

For reward $r_t = J(P_t; P_0) - J(P_{t+1}; P_0)$ and undiscounted finite horizon T,

$$\sum_{t=0}^{T-1} r_t = J(P_0; P_0) - J(P_T; P_0). \tag{5}$$

This identity requires a fixed cost/reference within the episode and $\gamma = 1$. Discounting, clipping, or extra penalties change the relationship. PPO's clipped policy-ratio objective constrains incentives for large updates, but does not guarantee transfer or physical legality (Schulman et al., 2017).

3.3 Hypotheses and proposed extension

H1: initializer robustness. Training on a mixture of starting-layout conditions reduces degradation on unfamiliar initializers compared with a single starting distribution.

H2: scaling sensitivity. A carefully defined transfer-reference convention may improve stability relative to native training-extrema scaling. Existing normalized coordinates and dimensions will be audited before adding redundant features.

H3: value beyond search. Any improvement remains useful relative to greedy search, simulated annealing, and continued analytical optimization under matched deployment budgets.

The core experiment is a 2×2 design: native versus proposed reference scaling, crossed with single versus mixed training starts. Hold network architecture, object selection, geometry, masks, objective terms, and training-step budget fixed. First vary legal perturbations and backbone checkpoints; introduce a distinct initializer only after its interface is verified. Perturbations change starting-state quality and are not themselves a new initializer algorithm. If scaling adds no meaningful distinction, prioritize the mixed-start study and stronger controls.

3.4 Role of diffusion and scope control

A diffusion comparison is a later extension. One experiment will compare complete generation-plus-refinement pipelines under a common downstream evaluator. Another will give every refiner the exact same diffusion-generated

placement. Include extra diffusion sampling and diffusion-plus-search controls so additional computation cannot masquerade as an RL advantage. Before this experiment, confirm macro IDs, net/pin definitions, dimensions, units, fixed objects, and objective values survive format conversion.

The recommended core is therefore *initialization-robust cross-circuit refinement under fixed compute budgets*. Adaptive regularity and learned stopping remain alternatives or extensions; implementing all directions is outside the intended eight-week scope.

4 Initial implementation and measured synthetic pilot

4.1 Implemented components

We implemented `prototype/placement_rl.py` using Python and NumPy. It contains a seeded synthetic-netlist generator, a placement environment, a vectorized weighted HPWL evaluator, geometric action masks, random and greedy controllers, a trainable policy, validation-based checkpoint selection, and CSV/JSON result logging. This extends the earlier hand-computed demonstration into an executable learning and evaluation scaffold.

Each synthetic circuit contains six rectangles on a 10×10 integer grid, two fixed terminals, ten initially sampled nets of degree two or three, and one central 2×2 blockage. Extra terminal connections are added if needed to connect an otherwise isolated macro. Rectangle dimensions are drawn from a fixed multiset of 1×1, 2×1, and 1×2 shapes. Nets have integer weights one or two. Each macro has a center pin; orientations are fixed.

The action set contains all currently feasible grid locations for the selected macro. A no-op is always available because the current position remains legal. Every transition preserves rectangle legality. This differs intentionally from the native MaskRegulate passwise mask, and does not reproduce halos, rows, standard cells, routing layers, or industrial pin geometry.

4.2 A small trainable refinement policy

For legal candidate a, define a five-dimensional observation $\phi(o_t, a)$ containing: scaled immediate normalized HPWL gain, negative normalized displacement, negative boundary-distance feature, gain multiplied by remaining-horizon fraction, and negative squared displacement. The boundary feature is an observation only; the prototype reward contains no regularity term. All methods have access to the same candidate evaluator, and greedy directly selects the minimum candidate cost.

The masked linear policy is

$$\pi_\theta(a \mid o_t) = \frac{\exp(\theta^\top \phi(o_t, a))}{\sum_{b \in \mathcal{A}(s_t)} \exp(\theta^\top \phi(o_t, b))}, \quad a \in \mathcal{A}(s_t). \tag{6}$$

Only five shared parameters are learned; they are independent of circuit identity. The policy is trained with REINFORCE (Williams, 1992), using

$$G_t = \sum_{k=t}^{T-1} r_k, \qquad \widehat{\nabla_\theta L} = \sum_{t=0}^{T-1} (G_t - b_t) \nabla_\theta \log \pi_\theta(a_t \mid o_t). \tag{7}$$

The baseline b_t is a lagged exponential moving average from previous episodes, updated after each trajectory. Gradients are averaged over ten episodes and norm-limited before an update of size 0.12. Training samples actions; evaluation chooses the maximum-probability action. This is REINFORCE, not PPO: there is no PPO clipped surrogate, learned critic, or native image encoder in the prototype.

The reward is $(H(P_t) - H(P_{t+1}))/H(P_0)$ with $\gamma = 1$. Episodes have twelve decisions, corresponding to two ordered passes through six macros. The training objective concerns terminal return. For deployment reporting, every controller retains its best feasible incumbent, including P_0. Terminal outcomes are reported separately so this selection does not conceal harmful trajectories.

4.3 Pilot setup and checks

The fixed split uses eight training netlists (generator seeds 101–108), two validation netlists (201–202), and four test netlists (301–304). Three independent policy-training seeds, 17, 29, and 43, each run 600 episodes. Training and validation starts are uniform feasible placements. Test starts use either that same distribution or an edge-biased

sequential sampler, with two start seeds per netlist. The edge sampler favors proximity to the nearest boundary; it is a distribution shift within this toy generator, not an industrial placement algorithm.

Checkpoints are chosen using uniform-validation gain at episode zero and every 100 episodes; ties retain the earliest checkpoint. Final test evaluation occurs only after all checkpoints are selected. Each test initializer has eight circuit/start cases. The policy has 24 runs per initializer because each case is evaluated with three trained seeds; the baselines have eight each. Policy-seed and start results are averaged within each netlist, then equally across the four netlists. No confidence claim about industrial families is made.

Checks compare vectorized candidate HPWL with an independent scalar calculation, reject overlapping/out-of-bounds placements, verify feasibility after every transition, and confirm the undiscounted reward telescoping identity. The pilot ran on CPU using NumPy; GPU access is reserved for the subsequent real-circuit work. Recorded training and validation time across the three seeds was 20.67 seconds on this host. Timing is diagnostic rather than a speed benchmark.

4.4 Measured results and interpretation

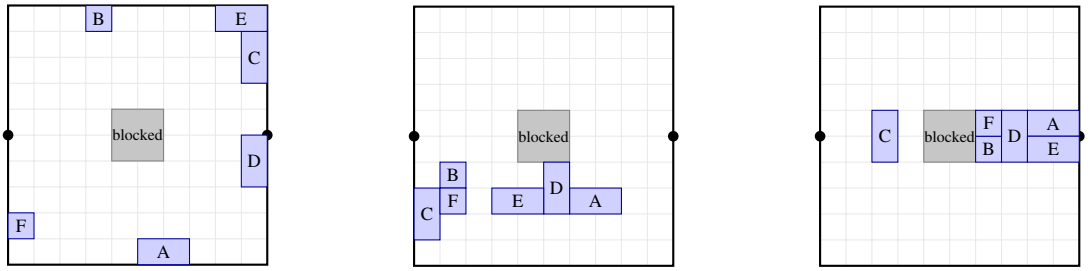
Table 3 is generated directly from the saved run files. Positive gain means $100[H(P_0) - H(P)]/H(P_0)$. All returned placements passed the implemented geometric checks. Each non-identity method receives twelve decisions; the number of feasible candidate evaluations varies by trajectory, so this is *not* an equal-wall-time or equal-objective-call experiment.

Table 3: Measured synthetic pilot on four held-out toy netlists. Selected gain uses the best feasible incumbent; terminal gain uses the final step. Harm is the fraction of terminal outcomes worse than the start. These values are not real-circuit or PPA results.

Test starts	Method	Selected gain (%)	Terminal gain (%)	Harm (%)
Uniform	No refinement	0.00	0.00	0.0
Uniform	Random masked	10.07	-8.21	50.0
Uniform	Greedy	57.88	57.88	0.0
Uniform	Learned policy	58.16	58.16	0.0
Edge-biased	No refinement	0.00	0.00	0.0
Edge-biased	Random masked	26.52	9.79	12.5
Edge-biased	Greedy	65.62	65.62	0.0
Edge-biased	Learned policy	67.09	67.09	0.0

The learned policy achieves 58.16% selected gain on uniform starts, versus 57.88% for greedy search. On edge-biased starts the corresponding gains are 67.09% and 65.62%. Both methods exploit extensive improvement headroom in small, unoptimized synthetic layouts. Their difference is modest, and four random netlists do not establish a reliable advantage. Larger edge-start gains may reflect weaker initial placements rather than better transfer. The result supports the operation of the learning/evaluation scaffold and the need for strong greedy controls.

Random actions illustrate why incumbent selection and terminal quality must be separated: retaining the best encountered layout can give positive selected gain even when the final trajectory is worse than the start. Incumbent selection is available to all methods and does not imply a new learning contribution.



Initial: $H = 180.5$ Greedy: $H = 63.0$ Learned policy: $H = 60.0$

Figure 1: A fixed illustrative test case: netlist seed 301, edge-biased start seed 8001, and policy seed 17. Rectangle labels identify the same six macros; black dots are fixed terminals and the gray square is blocked. The figure shows placement geometry, not routed wires. Weighted center-pin HPWL is computed from the saved netlist.

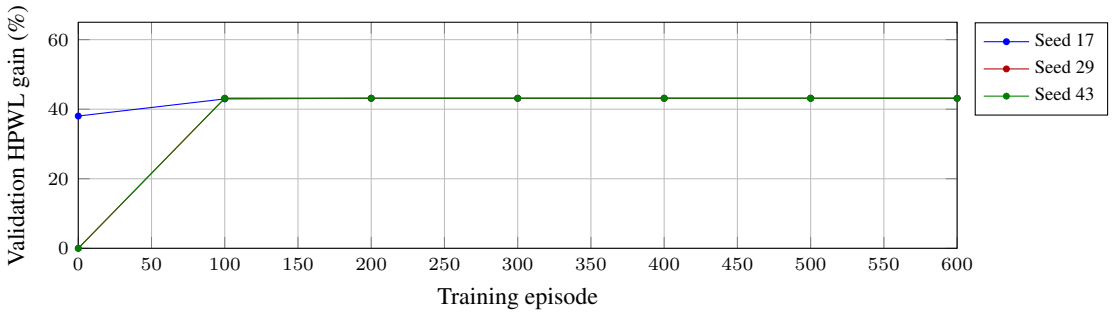


Figure 2: Recorded checkpoint-selection scores on the two validation netlists, averaged over two fixed uniform starts each. These curves show the synthetic training workflow; they are not final-test learning curves. Overlapping plateaus can reflect the limited capacity and simplicity of the toy problem.

4.5 What the pilot establishes and what it leaves open

The implementation demonstrates geometry-aware action generation, exact toy HPWL changes, trainable shared action preferences, validation-only model selection, and evaluation on new synthetic graphs and starting distributions. It does not implement the proposed mixed-start/scaling ablation, reproduce MaskRegulate, compare diffusion, or include real standard-cell placement and routing. Its exact candidate-cost features make greedy search particularly strong. Its compact observations may omit information needed for coordinated future moves, and all netlists share the same small generator family. These limitations determine the next implementation steps.

5 Planned evaluation and remaining work

5.1 Real-circuit experiment design

First establish one reproducible reference run and verify the parser, selected instances, geometry transformations, objective calculation, and placement export. The released reference selects modules by area, so this inventory must be reconciled with physical hard macros before comparing tools. Audit design lineage and checkpoint provenance before finalizing family-disjoint training, validation, and test splits.

Compare no refinement, random matched-mask actions, greedy search, tuned simulated annealing, the frozen original policy, and the proposed variants. Include continued analytical placement as a practical deployment control. Methods with different transition or legality semantics will be labeled separately from shared-mask controls. Simulated annealing will be tuned on validation only.

Run separate equal-wall-time and equal-objective-evaluation studies, including mask construction, inference, cost evaluation, legalization, candidate selection, and relevant backend calls. Native passwise methods may have no new feasible output before completing a pass. On interruption, return the last independently checked feasible endpoint or the common feasible start; no free post-deadline pass completion is allowed.

5.2 Metrics, ablations, and conclusions

Primary measurements are per-design HPWL improvement, feasibility, harmful-proposal frequency, completed passes, and quality versus time. Report macro-proxy and full standard-cell HPWL separately. Evaluate a declared congestion estimator, then attempt routing/timing on a small preselected compatible subset using a common backend (The OpenROAD Project, 2026). Power and area claims remain conditional on appropriate physical data; they cannot be inferred from HPWL.

The main ablation isolates native/proposed scaling and single/mixed starts. Use several training seeds when measured resource costs permit, pair test starts across methods, and aggregate at the design-family level. A failure to beat search, a loss under initializer shift, or disagreement between proxy and routing is a valid outcome if implementations, controls, and uncertainty are documented. Final-test outcomes will not be used to retune the method.

Table 4: Implementation status and eight-week continuation plan.

Stage	Work	Completion evidence
Completed at mid-sem	Literature/code review, problem formulation, synthetic environment, REINFORCE and controls	Source files, raw pilot runs, generated figures/tables and validation checks
Weeks 1–2	Native environment, one real circuit, provenance manifest and search interface	Repeated reference run, objective/geometry round-trip, measured memory/time
Weeks 3–4	Mixed-start training and scaling pilot	Controlled four-configuration validation; freeze final settings
Weeks 5–6	Held-out circuit/initializer tests and ablations	Shared inputs, budget accounting, failures and per-family tables
Weeks 7–8	Downstream subset, optional diffusion integration, analysis and report	Claims tied to saved artifacts; explicit limitations

6 Mid-semester assessment

The current work establishes a precise placement-refinement question, explains its relationship to constructive RL and generative placement, and provides an executable synthetic implementation. The pilot shows why a strong greedy baseline and transparent selection rules are necessary. The next objective is to test whether mixed starting conditions improve a frozen policy's usefulness on genuinely unseen circuit families under controlled budgets. The intended end-semester contribution is a reproducible robustness study and an explainable extension, supported by measured evidence rather than an assumed superiority of RL.

Reproducibility. The source, raw synthetic results, and research notes are maintained at https://github.com/BirariAditya/chip-refinement-generalization. The repository is private and requires access. The supplied source archive also includes the prototype and measured CSV/JSON files. Report figures and numeric tables are generated from those files. No real-circuit or PPA result is claimed in this report.

References

DREAMPlace contributors (2026). DREAMPlace: Official code and benchmark documentation. https://github.com/limbo018/DREAMPlace. Accessed 17 October 2026.

Gao, C., Shi, Y., Xue, K., Chen, R.-T., Xu, S., Yuan, M., Qian, C., and Zhou, Z.-H. (2026). Expertise can be helpful for reinforcement learning-based macro placement. In *International Conference on Learning Representations*.

Lai, Y., Liu, J., Tang, Z., Wang, B., Hao, J., and Luo, P. (2023). ChiPFormer: Transferable chip placement via offline decision transformer. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 18346–18364. PMLR.

Lai, Y., Mu, Y., and Luo, P. (2022). MaskPlace: Fast chip placement via reinforced visual representation learning. In *Advances in Neural Information Processing Systems*, volume 35, pages 24019–24030. Curran Associates, Inc.

LAMDA-BBO (2026). MaskRegulate: Official implementation. https://github.com/lamda-bbo/macro-regulator. Source inspected 17 October 2026, revision fe475473c86121bd09e031b6329f18e2d8b9d5ce.

Lee, V., Nguyen, M., Elzeiny, L., Deng, C., Abbeel, P., and Wawrzynek, J. (2025). Chip placement with diffusion models. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 33499–33514. PMLR.

Pu, Y., Chen, T., He, Z., Bai, C., Zheng, H., Lin, Y., and Yu, B. (2024). IncreMacro: Incremental macro placement refinement. In *Proceedings of the 2024 International Symposium on Physical Design*, ISPD '24, pages 169–176. Association for Computing Machinery.

Schulman, J., Wolski, F., Dhariwal, P., Radford, A., and Klimov, O. (2017). Proximal policy optimization algorithms. arXiv preprint arXiv:1707.06347.

The OpenROAD Project (2026). OpenROAD documentation. https://openroad.readthedocs.io/en/latest/. Accessed 17 October 2026.

Williams, R. J. (1992). Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 8:229–256.

Xie, P., Xue, K., Shi, Y., Chen, R.-T., Gao, C., Xu, S., Ding, C., Yuan, M., and Qian, C. (2026). FlowPlace: Flow matching for chip placement. In *Proceedings of the 63rd Design Automation Conference*.

Xue, K., Chen, R.-T., Lin, X., Shi, Y., Kai, S., Xu, S., and Qian, C. (2024). Reinforcement learning policy as macro regulator rather than macro placer. In *Advances in Neural Information Processing Systems*, volume 37, pages 140565–140588. Curran Associates, Inc.